

**create_
changelog_view**



Содержание

01

Обзор

Назначение и базовый синтаксис `create_changelog_view`

03

Опции чтения Spark

Snapshot-based и timestamp-based опции

05

Carry-over строки

Понятие и обработка carry-over rows

07

Net Changes

Режим чистых изменений без промежуточных состояний

02

Параметры

Подробное описание всех входных параметров

04

CDC метаданные

Специальные колонки `_change_type`, `_change_ordinal`

06

Pre/Post Update Images

Вычисление образов до/после обновления

08

Примеры использования

Snapshot-based и timestamp-based примеры

Обзор

🎯 Назначение

Процедура `create_changelog_view` создает представление (view), содержащее изменения строк между указанными снимками (snapshots) Iceberg таблицы. Это основной инструмент для реализации Change Data Capture (CDC) в Apache Iceberg.

✅ Возможности

- Отслеживание INSERT, DELETE, UPDATE
- Работа со снимками и временными метками
- Вычисление net changes
- Pre/post update images

⚠️ Ограничения

- Не поддерживает MOR таблицы
- Требуется Copy-on-Write режим
- Работает только в Spark

📌 Важно

Процедура использует `IncrementalChangelogScan` из Iceberg core модуля, который возвращает добавленные и логически удаленные файлы снимков Append, Overwrite и Delete. Основное ограничение — не возвращает delete файлы, что делает её непригодной для MOR (Merge-on-Read) таблиц.

</> Базовый синтаксис

```
-- Базовый вызов процедуры
CALL spark_catalog.system.create_changelog_view(
  table => 'db.tbl',
  options => map(
    'start-snapshot-id', '1',
    'end-snapshot-id', '2'
  )
);

-- Запрос созданного view
SELECT * FROM tbl_changes;
```

Параметры

Параметр	Обязательно	Тип	Описание
table	✓	string	Имя исходной таблицы Iceberg в формате 'db.tbl'. Полностью квалифицированное имя таблицы.
changelog_view	—	string	Имя создаваемого view. Если не указано, используется имя таблицы с суффиксом _changes.
options	—	map	Карта опций чтения Spark. Содержит параметры start-snapshot-id , end-snapshot-id , start-timestamp , end-timestamp .
net_changes	—	boolean	Выводить только чистые изменения (net changes), удаляя промежуточные состояния. По умолчанию false.
compute_updates	—	boolean	Вычислять pre/post update images. Преобразует пары DELETE/INSERT в UPDATE_BEFORE/UPDATE_AFTER. По умолчанию false.
identifier_columns	—	array	Список колонок-идентификаторов для вычисления updates. Используется при compute_updates=true. Если не указан, используются identifier fields таблицы.

Выходной параметр

Процедура возвращает [changelog_view](#) (string) — имя созданного представления.

Зависимости

[compute_updates](#) требует [identifier_columns](#) для определения пар DELETE/INSERT.

Опции чтения Spark



Snapshot-based

`start-snapshot-id`

Начальный snapshot ID (исключительно). Если не указан, чтение начинается с первого снимка таблицы включительно.

`end-snapshot-id`

Конечный snapshot ID (включительно). По умолчанию — текущий снимок таблицы.



Timestamp-based

`start-timestamp`

Начальная временная метка в миллисекундах (исключительно). Если не указана, чтение с первого снимка.

`end-timestamp`

Конечная временная метка в миллисекундах (включительно). По умолчанию — текущее время.



Примеры использования опций

-- Snapshot-based

```
options => map(
  'start-snapshot-id', '5390529835796506035',
  'end-snapshot-id', '8764748981452218370'
)
```

-- Timestamp-based

```
options => map(
  'start-timestamp', '1678335750489',
  'end-timestamp', '1678992105265'
)
```

-- Только end (от начала до указанного)

```
options => map(
  'end-snapshot-id', '8764748981452218370'
)
```



Совет

Snapshot-based подход более надежен, так как snapshot ID однозначно идентифицирует состояние таблицы. Timestamp-based удобен для работы с временными интервалами, но может быть менее точным при высокой частоте изменений.

CDC метаданные



`_change_type`

Тип изменения строки

INSERT

Новая строка была добавлена в таблицу

DELETE

Строка была удалена из таблицы

UPDATE_BEFORE

Состояние строки до обновления (требуется `compute_updates=true`)

UPDATE_AFTER

Состояние строки после обновления (требуется `compute_updates=true`)



`_change_ordinal`

Порядок изменений

Моноotonно возрастающее число для упорядочивания изменений в рамках одного диапазона снимков. Позволяет восстановить последовательность операций.



`_commit_snapshot_id`

ID снимка изменения

Идентификатор снимка (snapshot ID), в котором произошло изменение. Позволяет отследить, в какой транзакции была сделана модификация.



Пример результата

id	name	_change_type	_change_ord
1	Alice	INSERT	0
2	Bob	INSERT	0
1	Alice	DELETE	1
3	Carrol	UPDATE_BEFORE	2
3	Calypso	UPDATE_AFTER	2

Запрос:

```
SELECT * FROM tbl_changes
ORDER BY _change_ordinal;
```

Carry-over строки



Что такое Carry-over?

Carry-over строки — это артефакт операций **copy-on-write** (MERGE, UPDATE, DELETE). Когда происходит удаление строки из файла, содержащего несколько строк, Iceberg перезаписывает весь файл, сохраняя неизменные строки в новом файле.

Пример:

Файл содержит row1 (id=1, name='Alice') и row2 (id=2, name='Bob') . При удалении row2 файл перезаписывается, сохраняя row1. Changelog покажет DELETE row1 и INSERT row1, хотя row1 не изменялась!



Обработка по умолчанию

Процедура `create_changelog_view` **автоматически удаляет** carry-over строки. Это обеспечивает, что в changelog попадают только реальные изменения данных, а не артефакты copy-on-write механизма.



Просмотр Carry-over строк

Если вам нужно увидеть carry-over строки (например, для отладки), используйте прямой доступ к SparkChangelogTable :

```
-- Прямой запрос к changelog table  
SELECT *  
FROM spark_catalog.db.tbl.changes;
```

Pre/Post Update Images



Концепция

Pre/post update images позволяют видеть **состояние строки до и после обновления**. Вместо представления UPDATE как пары DELETE + INSERT, процедура создает две записи: UPDATE_BEFORE (старая версия) и UPDATE_AFTER (новая версия).

Требования:

- `compute_updates = true`
- `identifier_columns` — для определения пар DELETE/INSERT

</> Пример использования

```
-- Включение pre/post update images

CALL spark_catalog.system.create_changelog_view(
  table => 'db.tbl',
  options => map(...),
  compute_updates => true,
  identifier_columns => array('id')
);
```



Трансформация данных

Исходные данные (без `compute_updates`):

id	name	_change_type
3	Carrol	DELETE
3	Calvin	INSERT



С `compute_updates=true`:

id	name	_change_type
3	Carrol	UPDATE_BEFORE
3	Calvin	UPDATE_AFTER



Как это работает: Identifier columns используются для определения, что DELETE с `id=3` и INSERT с `id=3` относятся к одной строке.

Net Changes



Концепция Net Changes

Режим net_changes удаляет **промежуточные изменения** и оставляет только **финальное состояние** данных. Это полезно, когда нужно понять "что изменилось в итоге", а не отслеживать всю историю.

Пример сценария:

Строка вставлена в снимке 1 → обновлена в снимке 2 → удалена в снимке 3. С net_changes=true эта строка не появится в результате (нет net изменения).



Синтаксис

-- Включение net changes

```
CALL spark_catalog.system.create_changelog_view(  
  table => 'db.tbl',  
  options => map(...),  
  net_changes=> true,  
  identifier_columns => array('id')  
);
```



Пример эволюции

V0 → V1: INSERT id=1, val=1

id	val	type
1	1	INSERT

V1 → V2: UPDATE id=1, val=1→2

id	val	type
1	1	DELETE
1	2	INSERT



Net Changes (V0 → V2):

id	val	type
1	2	INSERT

Промежуточное значение 1 удалено

Примеры: Snapshot-based

БАЗОВЫЙ

```
-- Создание changelog view между снимками
CALL spark_catalog.system.create_changelog_view(
  table => 'db.tbl',
  options => map(
    'start-snapshot-id', '1',
    'end-snapshot-id', '2'
  )
);
```

Создает view с именем tbl_changes (по умолчанию) на основе изменений между снимками 1 (исключительно) и 2 (включительно).

С ИМЕНЕМ VIEW

```
-- С указанием имени changelog view
CALL spark_catalog.system.create_changelog_view(
  table => 'silver.orders',
  options => map(
    'start-snapshot-id', '1234567890',
    'end-snapshot-id', '9876543210'
  ),
  changelog_view => 'orders_cdc'
);
```

Создает view с именем orders_cdc для отслеживания изменений заказов.

C IDENTIFIER COLUMNS

```
-- Вычисление updates по колонкам-идентификаторам
CALL spark_catalog.system.create_changelog_view(
  table => 'db.customers',
  options => map(
    'start-snapshot-id', '1',
    'end-snapshot-id', '2'
  ),
  identifier_columns => array('customer_id', 'email')
);
```

Использует customer_id и email для определения, какие операции являются обновлениями одной и той же строки.



Запрос созданного view

```
-- Все изменения
SELECT * FROM orders_cdc;

-- Только INSERT с фильтрацией
SELECT * FROM orders_cdc
WHERE _change_type = 'INSERT'
AND order_amount > 1000
ORDER BY _change_ordinal;
```

После создания changelog view можно выполнять любые SQL-запросы для анализа изменений.

Примеры: Timestamp-based & Net Changes

TIMESTAMP-BASED

```
-- На основе временных меток
CALL spark_catalog.system.create_changelog_view(
  table => 'marketing.events',
  options => map(
    'start-timestamp', '1678335750489',
    'end-timestamp', '1678992105265'
  ),
  changelog_view => 'events_cdc'
);
```

Использует временные метки в миллисекундах (Unix timestamp) для определения диапазона. Удобно для интеграции с внешними системами.

ТОЛЬКО END

```
-- От начала таблицы до указанного снимка
CALL spark_catalog.system.create_changelog_view(
  table => 'db.tbl',
  options => map(
    'end-snapshot-id', '8764748981452218370'
  )
);
```

Если не указан start, чтение начинается с первого снимка таблицы. Полезно для начальной загрузки данных.

NET CHANGES

```
-- Чистые изменения без промежуточных состояний
CALL spark_catalog.system.create_changelog_view(
  table => 'db.tbl',
  options => map(
    'end-snapshot-id', '8764748981452218370'
  ),
  net_changes => true
);
```

Удаляет промежуточные изменения. Если строка была вставлена в снимке 1, обновлена в снимке 2 и удалена в снимке 3 — в результате её не будет (нет net изменения).



Как работают Net Changes

- Объединяет изменения across multiple snapshots
- Удаляет промежуточные состояния (INSERT→UPDATE→DELETE = нет изменения)
- UPDATE представлен как DELETE + INSERT пары
- Используйте, когда не требуется история промежуточных изменений